

noiseEST

Qun Li

20 Oct 2025 at 15:06:30

Contents

1	envs	1
2	libraries	1
3	def function	2
4	How to generate random data	3
5	loadData	4
6	function to count mut cell and read	6
7	Mouse background	14
8	Mouse somatic number	16
9	cluster mismatches	18
10	sessionInfo	25

1 envs

```
rm(list = ls())  
setwd("/Users/liqun/Desktop/scMutrace-tutorial/ReproducibilityAnalysis/Figure1/")
```

2 libraries

```

library(ggplot2)
library(reshape2)
library(patchwork)
library(fitdistrplus)
## Loading required package: MASS
##
## Attaching package: 'MASS'
## The following object is masked from 'package:patchwork':
##
##     area
## Loading required package: survival
library(goftest)
library(VGAM)
## Loading required package: stats4
## Loading required package: splines
library(dplyr)
##
## Attaching package: 'dplyr'
## The following object is masked from 'package:MASS':
##
##     select
## The following objects are masked from 'package:stats':
##
##     filter, lag
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
library(extraDistr)
##
## Attaching package: 'extraDistr'
## The following objects are masked from 'package:VGAM':
##
##     dfrechet, dgev, dgompertz, dgpdp, dgumbel, dhuber, dkumar, dlaplace,
##     dlomax, dpareto, drayleigh, dskellam, dslash, pfrechet, pgev,
##     pgompertz, pgpd, pgumbel, phuber, pkumar, plaplace, plomax,
##     ppareto, prayleigh, pslash, qfrechet, qgev, qgompertz, qgpdp,
##     qgumbel, qhuber, qkumar, qlaplace, qlomax, qpareto, qrayleigh,
##     rfrechet, rgev, rgompertz, rgpd, rgumbel, rhuber, rkumar, rlaplace,
##     rlomax, rpareto, rrayleigh, rskellam, rslash

```

3 def function

```

random_pick_count <- function(dataInput, sampleSize = 1000) {
  sample_data <- dataInput[sample(nrow(dataInput), min(sampleSize, nrow(dataInput))), ]

  # Cell rate
  mut_cells_rate_original <- 1 - sum(sample_data$altCellNum_original == 0) / sampleSize
  mut_cells_rate_simple <- 1 - sum(sample_data$altCellNum_simple1000 == 0) / sampleSize
  mut_cells_rate_scMutrace <- 1 - sum(sample_data$altCellNum_feature1000 == 0) / sampleSize

  # Read rate

```

```

mut_reads_rate_original <- sum(sample_data$reads_alt_original) /
  (sum(sample_data$reads_ref_original) + sum(sample_data$reads_alt_original))
mut_reads_rate_simple <- sum(sample_data$reads_alt_simple1000) /
  (sum(sample_data$reads_ref_simple1000) + sum(sample_data$reads_alt_simple1000))
mut_reads_rate_scMutrace <- sum(sample_data$reads_alt_feature1000) /
  (sum(sample_data$reads_ref_feature1000) + sum(sample_data$reads_alt_feature1000))

# Site rate
mut_sites_rate_original <- 1 - mean(sample_data$reads_alt_original <= 1)
mut_sites_rate_simple <- 1 - mean(sample_data$reads_alt_simple1000 <= 1)
mut_sites_rate_scMutrace <- 1 - mean(sample_data$reads_alt_feature1000 <= 1)

res <- data.frame(
  mutCell_original = mut_cells_rate_original,
  mutCell_simple = mut_cells_rate_simple,
  mutCell_scMutrace = mut_cells_rate_scMutrace,
  mutRead_original = mut_reads_rate_original,
  mutRead_simple = mut_reads_rate_simple,
  mutRead_scMutrace = mut_reads_rate_scMutrace,
  mutSite_original = mut_sites_rate_original,
  mutSite_simple = mut_sites_rate_simple,
  mutSite_scMutrace = mut_sites_rate_scMutrace
)

return(res)
}

```

4 How to generate random data

```

# Example
#####
# Step 1: randomly pick 100,000 lines from vcf file without repeat
#####
# vcfIn="/path/to/your/vcf/file.vcf" # output of scMutrace vcf file
# python random_pick_lines_no_repeat.py ${vcfIn} 100000 ${vcfIn}_random_pick

#####
# Step 2: estimate random background from the random picked vcf file
#####
# outDir="/path/to/your/output/directory/"
# bamIn="/path/to/your/bam/file.bam" # input bam file
# genome.fa="/path/to/your/reference/genome.fa" # reference genome fasta file
# sample="sampleID" # sample ID
# python 0_runStep2_EstimateCluster.py --outdir ${outDir} --bam ${bamIn}
#       --ref ${genome.fa} --vcfpath ${vcfIn}_random_pick --id ${sample}
#       --binsize 5 --dpsize 5 --altdpsize 3 --cellsize 5 --altcellsize 2
#       --min_bq 20 --min_mq 20 --cbtag CB --nprocs 10

```

5 loadData

```
# define col names
col_Names_data <- c("base_quality_original",
                    "base_quality_simple100", "base_quality_simple1000",
                    "base_quality_simple100Wei", "base_quality_simple1000Wei",
                    "base_quality_feature100", "base_quality_feature1000",
                    "reads_ref_original", "reads_alt_original", "reads_num_original",
                    "reads_ref_simple100", "reads_alt_simple100", "reads_num_simple100",
                    "reads_ref_simple1000", "reads_alt_simple1000", "reads_num_simple1000",
                    "reads_ref_simple100Wei", "reads_alt_simple100Wei", "reads_num_simple100Wei",
                    "reads_ref_simple1000Wei", "reads_alt_simple1000Wei", "reads_num_simple1000Wei",
                    "reads_ref_feature100", "reads_alt_feature100", "reads_num_feature100",
                    "reads_ref_feature1000", "reads_alt_feature1000", "reads_num_feature1000",
                    "totalCellsNum_original_raw",
                    "altCellNum_original", "totalCellsNum_original",
                    "altCellNum_simple100", "totalCellsNum_simple100",
                    "altCellNum_simple1000", "totalCellsNum_simple1000",
                    "altCellNum_simple100Wei", "totalCellsNum_simple100Wei",
                    "altCellNum_simple1000Wei", "totalCellsNum_simple1000Wei",
                    "altCellNum_feature100", "totalCellsNum_feature100",
                    "altCellNum_feature1000", "totalCellsNum_feature1000")

# define file tags
file_tags <- list(
  SMART_ERCC_random = "Mouse_SMART_ERCC",
  SMART_random = "Mouse_SMART_random",
  P16358_6001_random = "Mouse_Tenx_random",
  AML16_DX_scATA_random = "Human_Tenx_scATAC_DX_random",
  AML16_REM_scATA_random = "Human_Tenx_scATAC_REM_random",
  AML16_REL_scATA_random = "Human_Tenx_scATAC_REL_random",
  AML16_DX_scrNA_random = "Human_Tenx_scrNA_DX_random",
  AML16_REM_scrNA_random = "Human_Tenx_scrNA_REM_random",
  AML16_REL_scrNA_random = "Human_Tenx_scrNA_REL_random",
  EPSRC1_scrNA_random = "EPSRC1_Tenx_scrNA_random",
  EPSRC2_scrNA_random = "EPSRC2_Tenx_scrNA_random",
  EPSRC4_scrNA_random = "EPSRC4_Tenx_scrNA_random",
  EPSRC1_scATA_random = "EPSRC1_Tenx_scATA_random",
  EPSRC2_scATA_random = "EPSRC2_Tenx_scATA_random",
  EPSRC4_scATA_random = "EPSRC4_Tenx_scATA_random",
  Tenx_ERCC_random = "Tenx_ERCC_random"
)

# load data
allDataList <- lapply(names(file_tags), function(name) {
  message(name)
  file_path <- file.path("../Data/RandomData", paste0(name, ".est.random"))
  dat <- read.table(file_path, header = FALSE, sep = "\t")
  colnames(dat) <- col_Names_data
  dat$Tag <- file_tags[[name]]
  return(dat)
})
## SMART_ERCC_random
```

```

## SMART_random
## P16358_6001_random
## AML16_DX_scATA_random
## AML16_REM_scATA_random
## AML16_REL_scATA_random
## AML16_DX_scrNA_random
## AML16_REM_scrNA_random
## AML16_REL_scrNA_random
## EPSRC1_scrNA_random
## EPSRC2_scrNA_random
## EPSRC4_scrNA_random
## EPSRC1_scATA_random
## EPSRC2_scATA_random
## EPSRC4_scATA_random
## Tenx_ERCC_random

# merge
data_merge_noise <- do.call(rbind, allDataList)

data_Mut_quality <- data_merge_noise[,c("base_quality_original", "base_quality_simple1000", "base_quali

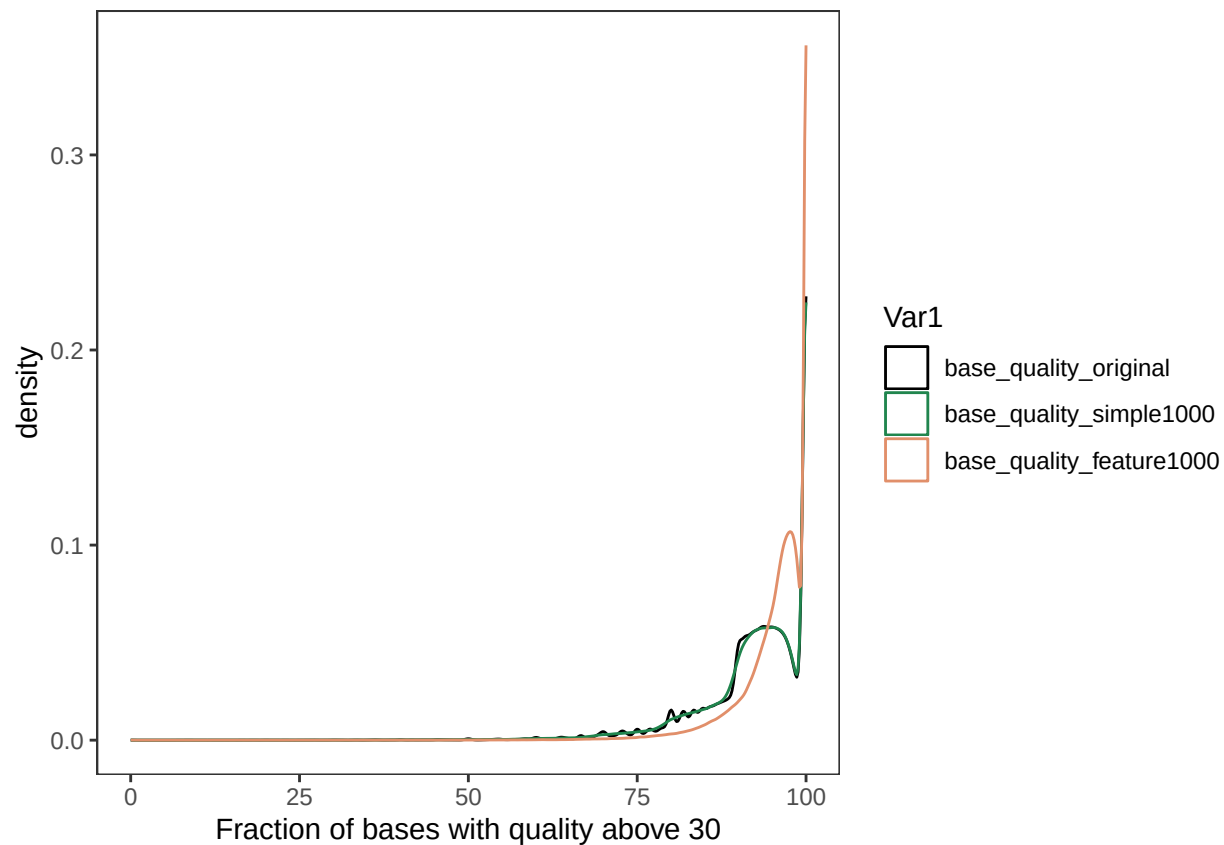
data_Mut_quality_re <- melt(t(data_Mut_quality))

colorAssign_1 = c(base_quality_original = "black",
                  base_quality_simple100 = "#e5f5e0", base_quality_simple1000 = "#20854EFF",
                  base_quality_simple100Wei = "#9ecae1", base_quality_simple1000Wei = "#084594",
                  base_quality_feature100 = "#fee5d9", base_quality_feature1000 = "#E18F6A")

p_mut_basequality <- ggplot(data_Mut_quality_re) +
  xlab("Fraction of bases with quality above 30") +
  geom_density(mapping = aes(x = 100 - value, color = Var1), show.legend = TRUE, alpha = 0.3) +
  scale_color_manual(values = colorAssign_1) +
  geom_rug() + theme_bw() + theme(panel.grid=element_blank())

p_mut_basequality

```



6 function to count mut cell and read

```
# function to count mut cell and read
count_mutCellRead <- function(data_merge_Input, sampleSize = 1000, n_iter = 1000) {
  set.seed(2025)

  res_list <- vector("list", n_iter)

  for (i in seq_len(n_iter)) {
    tmp_res <- random_pick_count(data_merge_Input, sampleSize)
    tmp_res$iteration <- i
    res_list[[i]] <- tmp_res
  }

  data_Plot <- dplyr::bind_rows(res_list)
  return(data_Plot)
}

# merge all data
uniqueTags <- unique(data_merge_noise$Tag)
plot_list <- list()

for (tag in uniqueTags) {
```

```

message(paste0("Processing ", tag))
tmp_data <- data_merge_noise %>% dplyr::filter(Tag == tag)
tmp_plot <- count_mutCellRead(tmp_data, sampleSize = 1000, n_iter = 1000)
tmp_plot$Tag <- tag
plot_list[[tag]] <- tmp_plot
}

## Processing Mouse_SMART_ERCC
## Processing Mouse_SMART_random
## Processing Mouse_Tenx_random
## Processing Human_Tenx_scATAC_DX_random
## Processing Human_Tenx_scATAC_REM_random
## Processing Human_Tenx_scATAC_REL_random
## Processing Human_Tenx_scRNA_DX_random
## Processing Human_Tenx_scRNA_REM_random
## Processing Human_Tenx_scRNA_REL_random
## Processing EPSRC1_Tenx_scRNA_random
## Processing EPSRC2_Tenx_scRNA_random
## Processing EPSRC4_Tenx_scRNA_random
## Processing EPSRC1_Tenx_scATA_random
## Processing EPSRC2_Tenx_scATA_random
## Processing EPSRC4_Tenx_scATA_random
## Processing Tenx_ERCC_random

data_merge_noiseEST_Plot <- do.call(rbind, plot_list)

```

```

p_background_est_read <- ggplot(
  data_merge_noiseEST_Plot[
    data_merge_noiseEST_Plot$Tag %in% c(
      "Human_Tenx_scATAC_DX_random",
      "Human_Tenx_scRNA_DX_random",
      "Human_Tenx_scATAC_REM_random",
      "Human_Tenx_scRNA_REM_random",
      "Human_Tenx_scATAC_REL_random",
      "Human_Tenx_scRNA_REL_random",
      "EPSRC1_Tenx_scRNA_random",
      "EPSRC2_Tenx_scRNA_random",
      "EPSRC4_Tenx_scRNA_random",
      "EPSRC1_Tenx_scATA_random",
      "EPSRC2_Tenx_scATA_random",
      "EPSRC4_Tenx_scATA_random"
    ),],
  aes(
    x = mutRead_scMutrace,
    color = Tag,
    linetype = ifelse(grepl("scRNA", Tag), "RNA", "ATAC")
  )
) +
  geom_density(aes(alpha = 0.4)) +
  theme_classic(base_size = 14) +
  labs(
    x = "Mutation Read Ratio",
    y = "Density",
    title = "Density plot of Mutation Read Ratio by Tag"
  )

```

```

) +
scale_color_manual(
  values = c(
    "Human_Tenx_scATAC_DX_random" = "#33a02c",
    "Human_Tenx_scRNA_DX_random" = "#33a02c",
    "Human_Tenx_scATAC_REM_random" = "#1f78b4",
    "Human_Tenx_scRNA_REM_random" = "#1f78b4",
    "Human_Tenx_scATAC_REL_random" = "#e31a1c",
    "Human_Tenx_scRNA_REL_random" = "#e31a1c",
    "EPSRC1_Tenx_scRNA_random" = "#ff7f00",
    "EPSRC2_Tenx_scRNA_random" = "#6a3d9a",
    "EPSRC4_Tenx_scRNA_random" = "#b15928",
    "EPSRC1_Tenx_scATA_random" = "#ff7f00",
    "EPSRC2_Tenx_scATA_random" = "#6a3d9a",
    "EPSRC4_Tenx_scATA_random" = "#b15928"
  )
) +
scale_linetype_manual(
  values = c("RNA" = "solid", "ATAC" = "dashed")
) +
theme(
  plot.title = element_text(hjust = 0.5),
  legend.position = "right",
  legend.title = element_blank()
)

p_background_est_read_aveHuman <- ggplot(
  data_merge_noiseEST_Plot[
    data_merge_noiseEST_Plot$Tag %in% c(
      "Human_Tenx_scATAC_DX_random",
      "Human_Tenx_scRNA_DX_random",
      "Human_Tenx_scATAC_REM_random",
      "Human_Tenx_scRNA_REM_random",
      "Human_Tenx_scATAC_REL_random",
      "Human_Tenx_scRNA_REL_random"
    ),],
  aes(
    x = mutRead_scMutrace
  )
) +
geom_density(aes(alpha = 0.4)) +
theme_classic(base_size = 14) +
labs(
  x = "Mutation Read Ratio",
  y = "Density",
  title = "Density plot of Mutation Read Ratio by Tag"
) +
theme(
  plot.title = element_text(hjust = 0.5),
  legend.position = "right",
  legend.title = element_blank()
)

```



```

p_background_est_cell <- ggplot(
  data_merge_noiseEST_Plot[
    data_merge_noiseEST_Plot$Tag %in% c(
      "Human_Tenx_scATAC_DX_random",
      "Human_Tenx_scRNA_DX_random",
      "Human_Tenx_scATAC_REM_random",
      "Human_Tenx_scRNA_REM_random",
      "Human_Tenx_scATAC_REL_random",
      "Human_Tenx_scRNA_REL_random",
      "EPSRC1_Tenx_scRNA_random",
      "EPSRC2_Tenx_scRNA_random",
      "EPSRC4_Tenx_scRNA_random",
      "EPSRC1_Tenx_scATA_random",
      "EPSRC2_Tenx_scATA_random",
      "EPSRC4_Tenx_scATA_random"
    ),],
  aes(
    x = mutCell_scMutrace,
    color = Tag,
    linetype = ifelse(grepl("scRNA", Tag), "RNA", "ATAC")
  )
) +
  geom_density(aes(alpha = 0.4)) +
  theme_classic(base_size = 14) +
  labs(
    x = "Mutation Cell Ratio",
    y = "Density",
    title = "Density plot of Mutation Cell Ratio by Tag"
  ) +
  scale_color_manual(
    values = c(
      "Human_Tenx_scATAC_DX_random" = "#33a02c",
      "Human_Tenx_scRNA_DX_random" = "#33a02c",
      "Human_Tenx_scATAC_REM_random" = "#1f78b4",
      "Human_Tenx_scRNA_REM_random" = "#1f78b4",
      "Human_Tenx_scATAC_REL_random" = "#e31a1c",
      "Human_Tenx_scRNA_REL_random" = "#e31a1c",
      "EPSRC1_Tenx_scRNA_random" = "#ff7f00",
      "EPSRC2_Tenx_scRNA_random" = "#6a3d9a",
      "EPSRC4_Tenx_scRNA_random" = "#b15928",
      "EPSRC1_Tenx_scATA_random" = "#ff7f00",
      "EPSRC2_Tenx_scATA_random" = "#6a3d9a",
      "EPSRC4_Tenx_scATA_random" = "#b15928"
    )
  ) +
  scale_linetype_manual(
    values = c("RNA" = "solid", "ATAC" = "dashed")
  ) +
  theme(
    plot.title = element_text(hjust = 0.5),
    legend.position = "right",
    legend.title = element_blank()
  )

```

```

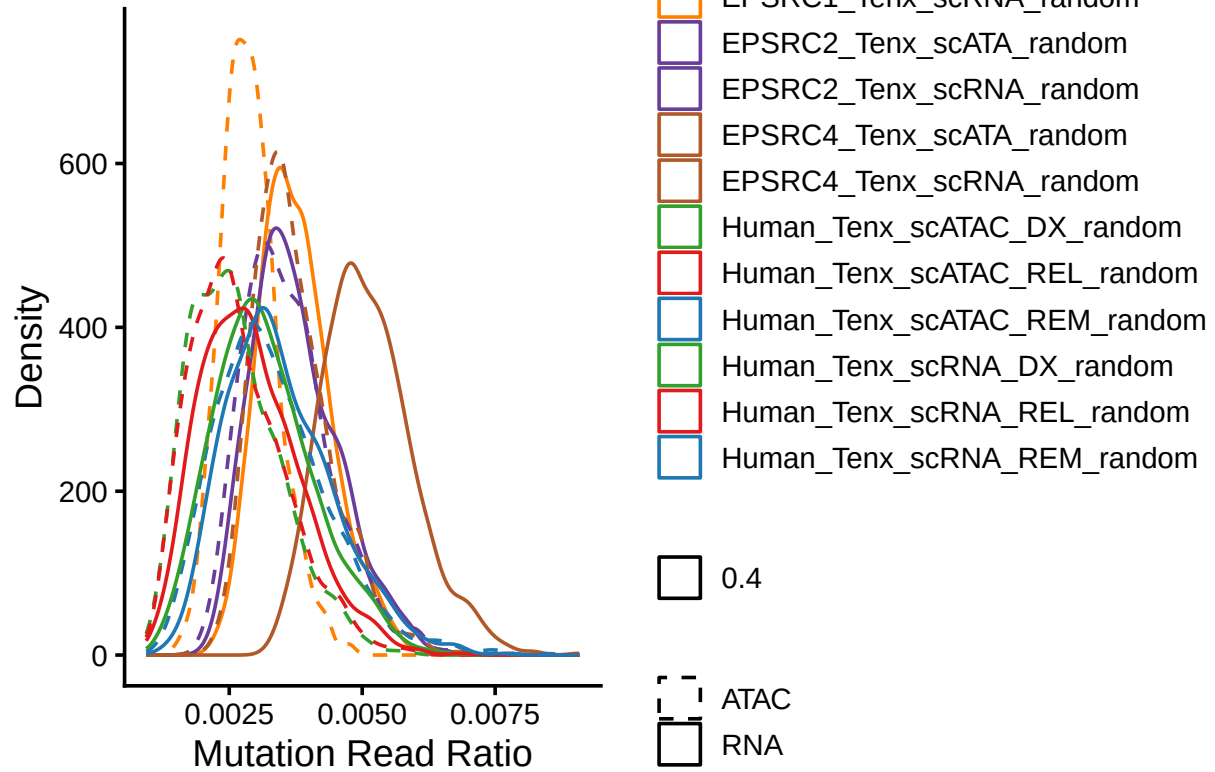
)

p_background_est_cell_aveHuman <- ggplot(
  data_merge_noiseEST_Plot[
    data_merge_noiseEST_Plot$Tag %in% c(
      "Human_Tenx_scATAC_DX_random",
      "Human_Tenx_scRNA_DX_random",
      "Human_Tenx_scATAC_REM_random",
      "Human_Tenx_scRNA_REM_random",
      "Human_Tenx_scATAC_REL_random",
      "Human_Tenx_scRNA_REL_random"
    ),],
  aes(
    x = mutCell_scMutrace
  )
) +
  geom_density(aes(alpha = 0.4)) +
  theme_classic(base_size = 14) +
  labs(
    x = "Mutation Cell Ratio",
    y = "Density",
    title = "Density plot of Mutation Cell Ratio by Tag"
  ) +
  theme(
    plot.title = element_text(hjust = 0.5),
    legend.position = "right",
    legend.title = element_blank()
  )

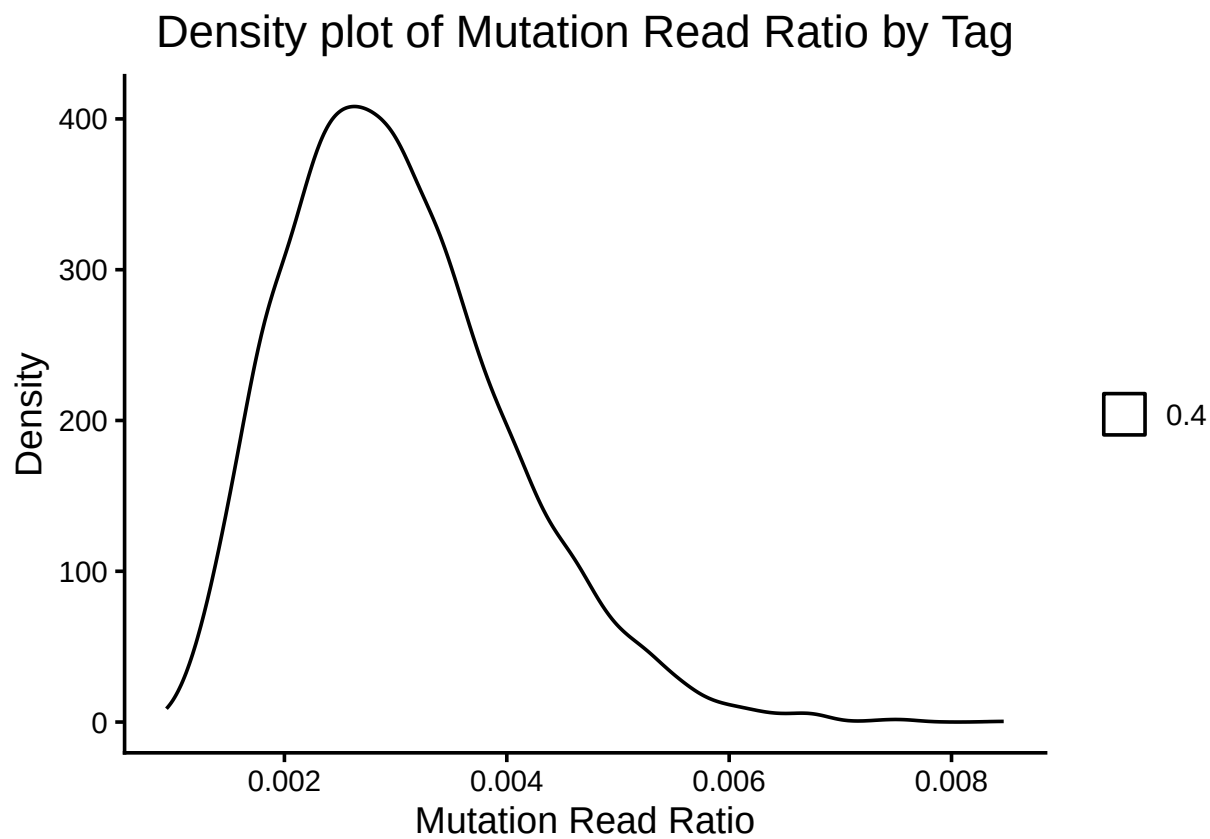
p_background_est_read

```

Density plot of Mutation Read Ratio by Tag

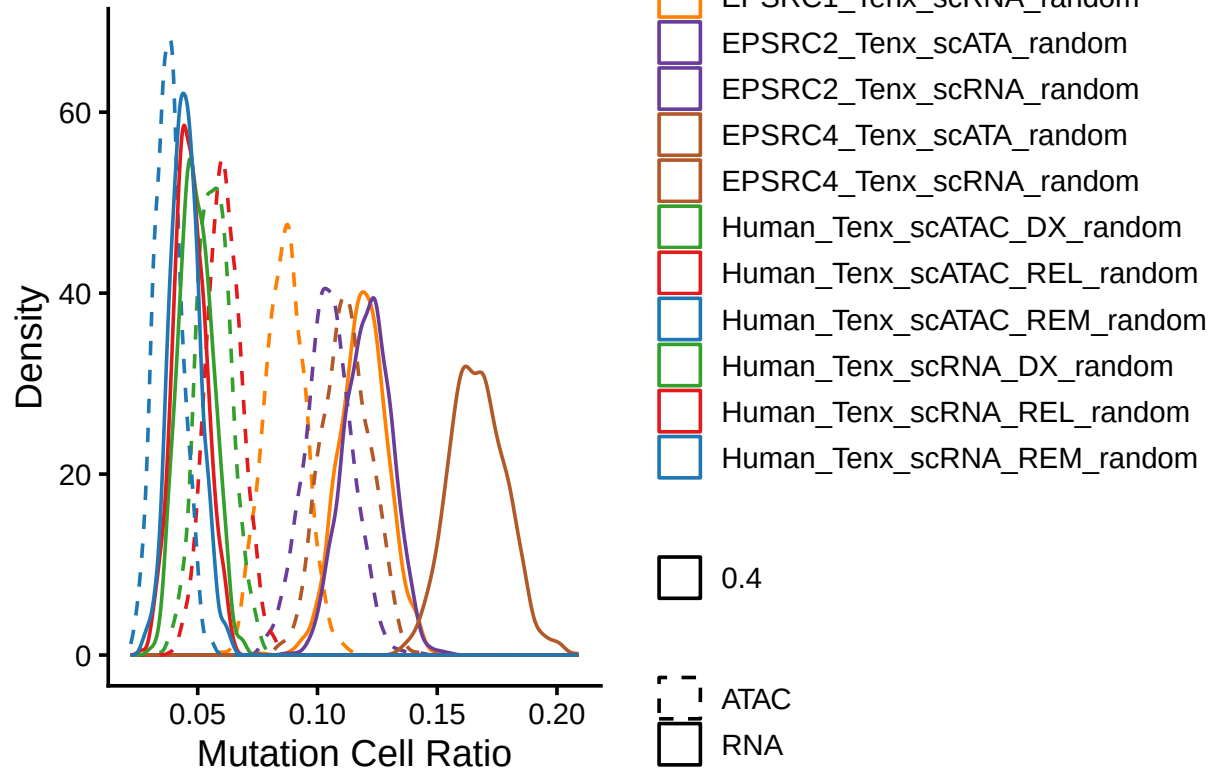


p_background_est_read_aveHuman



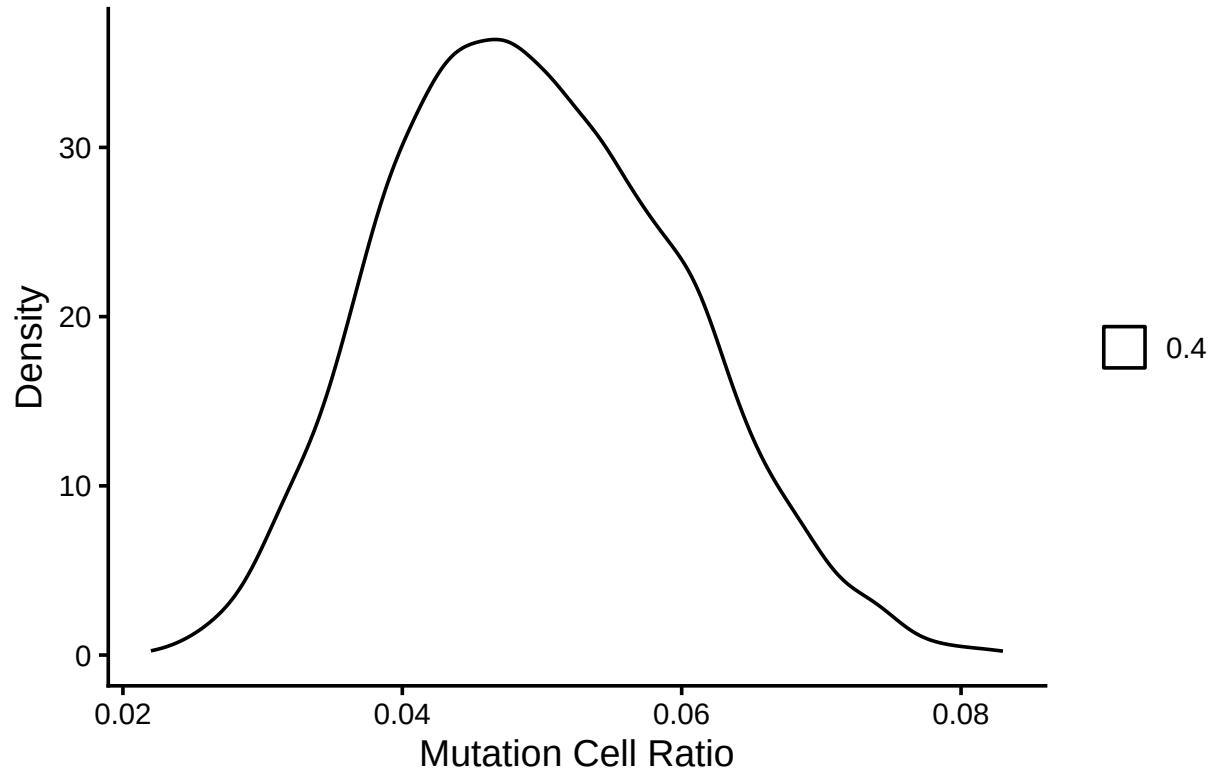
p_background_est_cell

ensity plot of Mutation Cell Ratio by Tag



p_background_est_cell_aveHuman

Density plot of Mutation Cell Ratio by Tag



```
Example_Human <- data_merge_noiseEST_Plot[
  data_merge_noiseEST_Plot$Tag %in% c(
    "Human_Tenx_scATAC_DX_random",
    "Human_Tenx_scRNA_DX_random",
    "Human_Tenx_scATAC_REM_random",
    "Human_Tenx_scRNA_REM_random",
    "Human_Tenx_scATAC_REL_random",
    "Human_Tenx_scRNA_REL_random"
  ),]

quantile(Example_Human$mutRead_scMutrace, probs = c(0.10, 0.25, 0.5, 0.75, 0.90))
##      10%      25%      50%      75%      90%
## 0.001817819 0.002285135 0.002898516 0.003619503 0.004382699
quantile(Example_Human$mutCell_scMutrace, probs = c(0.10, 0.25, 0.5, 0.75, 0.90))
##      10%      25%      50%      75%      90%
## 0.037 0.042 0.049 0.057 0.063
```

7 Mouse background

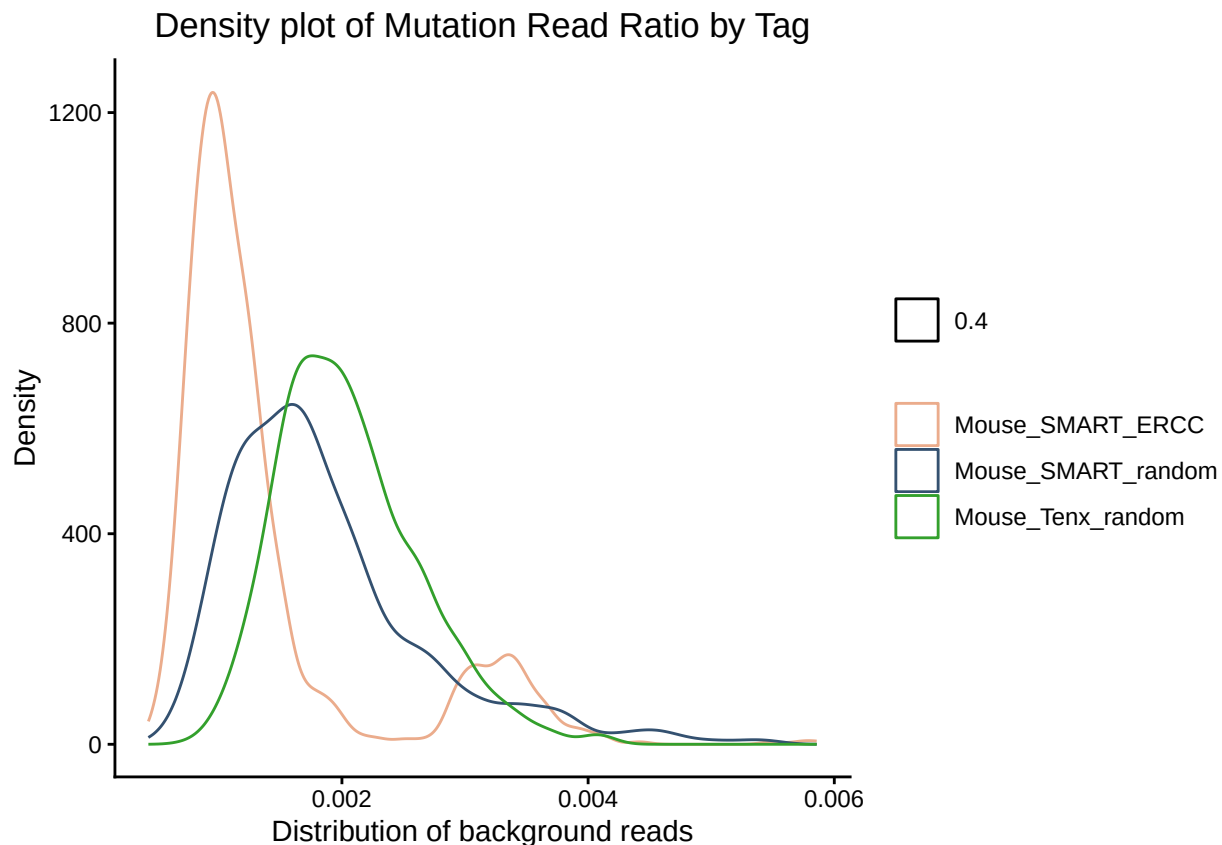
```
p_background_est_read_mouse <- ggplot(
  data_merge_noiseEST_Plot[
    data_merge_noiseEST_Plot$Tag %in% c(
      "Mouse_SMART_ERCC",
```

```

      "Mouse_SMART_random",
      "Mouse_Tenx_random"
    ),],
    aes(
      x = mutRead_scMutrace,
      color = Tag
    )
  ) +
  geom_density(aes(alpha = 0.4)) +
  theme_classic() +
  labs(
    x = "Distribution of background reads",
    y = "Density",
    title = "Density plot of Mutation Read Ratio by Tag"
  ) +
  scale_color_manual(
    values = c(
      "Mouse_SMART_ERCC" = "#EBAC8C",
      "Mouse_SMART_random" = "#345170",
      "Mouse_Tenx_random" = "#33a02c"
    )
  ) +
  theme(
    plot.title = element_text(hjust = 0.5),
    legend.position = "right",
    legend.title = element_blank()
  )
)

p_background_est_read_mouse

```



8 Mouse somatic number

```

Mouse_tenx_csm <- readRDS("../Data/Mouse/Mouse_tenx_csm.rds")
Mouse_tenx_meta <- readRDS("../Data/Mouse/Mouse_tenx_meta.rds")

Mouse_smar_csm <- as.data.frame(t(readRDS("../Data/Mouse/Mouse_smart_csm.rds")))
Mouse_smar_meta <- readRDS("../Data/Mouse/Mouse_smart_meta.rds")

data_MutPerCell_tenx <- data.frame(
  CellTag = rownames(Mouse_tenx_csm),
  MutNum_somatic = rowSums(Mouse_tenx_csm == "Mut"),
  WTNuM_somatic = rowSums(Mouse_tenx_csm == "WT")
)

data_MutPerCell_smart <- data.frame(
  CellTag = rownames(Mouse_smar_csm),
  MutNum_somatic = rowSums(Mouse_smar_csm == "Mut"),
  WTNuM_somatic = rowSums(Mouse_smar_csm == "WT")
)

data_MutPerCell_smart$Barcode <- data_MutPerCell_smart$CellTag
data_MutPerCell_smart_meta <- merge(data_MutPerCell_smart, Mouse_smar_meta, by = "Barcode")

```



```

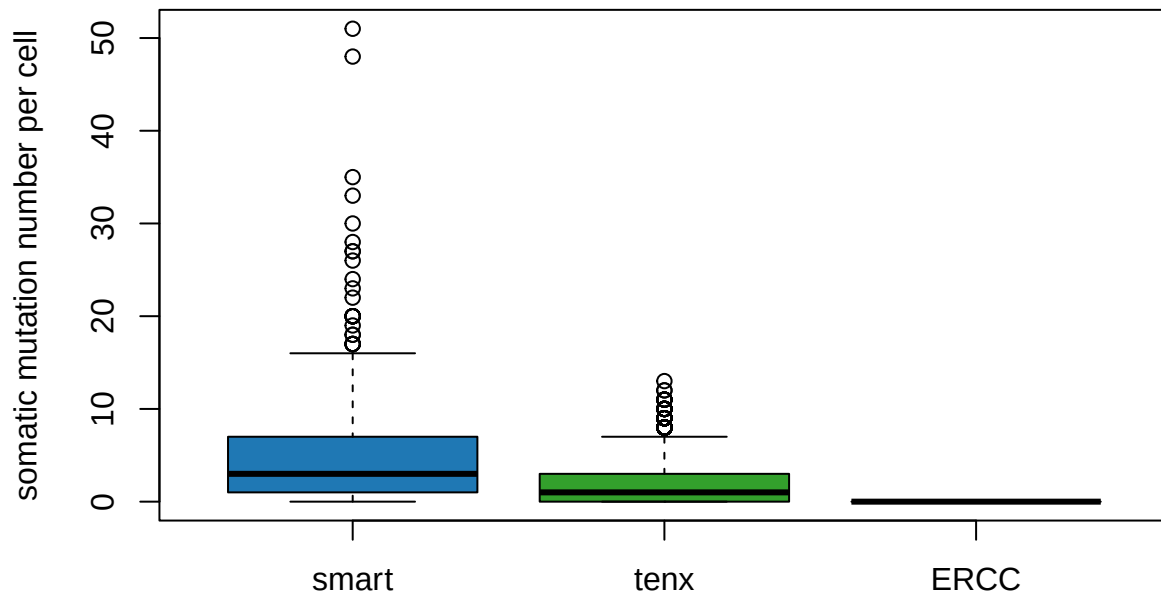
data_CellPerMut_tenx <- data.frame(
  Location = colnames(Mouse_tenx_csm),
  CellMutNum = colSums(Mouse_tenx_csm == "Mut"),
  CellWTNum = colSums(Mouse_tenx_csm == "WT")
)

data_CellPerMut_smart <- data.frame(
  Location = colnames(Mouse_smar_csm),
  CellMutNum = colSums(Mouse_smar_csm == "Mut"),
  CellWTNum = colSums(Mouse_smar_csm == "WT")
)

ERCC <- c(rep(0, 100))

boxplot(list(smart = data_CellPerMut_smart$CellMutNum,
             tenx = data_CellPerMut_tenx$CellMutNum,
             ERCC = ERCC),
  ylab = "somatic mutation number per cell",
  col = c("#1f78b4", "#33a02c", "#6a3d9a"))

```



9 cluster mismatches

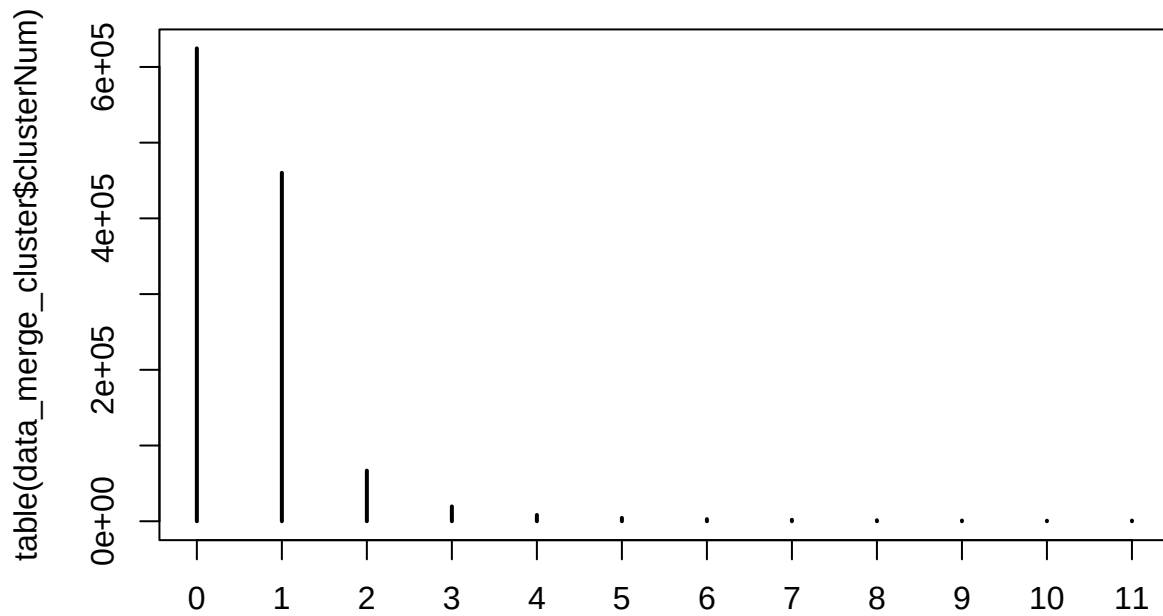
```
col_Names_cluster <- c("chr", "pos", "clusterNum")

file_tags <- list(
  AML16_DX_scATA = "Human_scATAC_DX_cluster",
  AML16_REM_scATA = "Human_scATAC_REM_cluster",
  AML16_REL_scATA = "Human_scATAC_REL_cluster",
  AML16_DX_scRNA = "Human_scRNA_DX_cluster",
  AML16_REM_scRNA = "Human_scRNA_REM_cluster",
  AML16_REL_scRNA = "Human_scRNA_REL_cluster",
  EPSRC1_scRNA = "EPSRC1_scRNA_cluster",
  EPSRC2_scRNA = "EPSRC2_scRNA_cluster",
  EPSRC4_scRNA = "EPSRC4_scRNA_cluster",
  EPSRC1_scATA = "EPSRC1_scATA_cluster",
  EPSRC2_scATA = "EPSRC2_scATA_cluster",
  EPSRC4_scATA = "EPSRC4_scATA_cluster"
)

allDataList_cluster <- lapply(names(file_tags), function(name) {
  file_path <- file.path("../Data/randomCluster/", paste0(name, ".scmutrace.cluster.count"))
  dat <- read.table(file_path, header = FALSE, sep = "\t")
  colnames(dat) <- col_Names_cluster
  dat$Tag <- file_tags[[name]]
  return(dat)
})

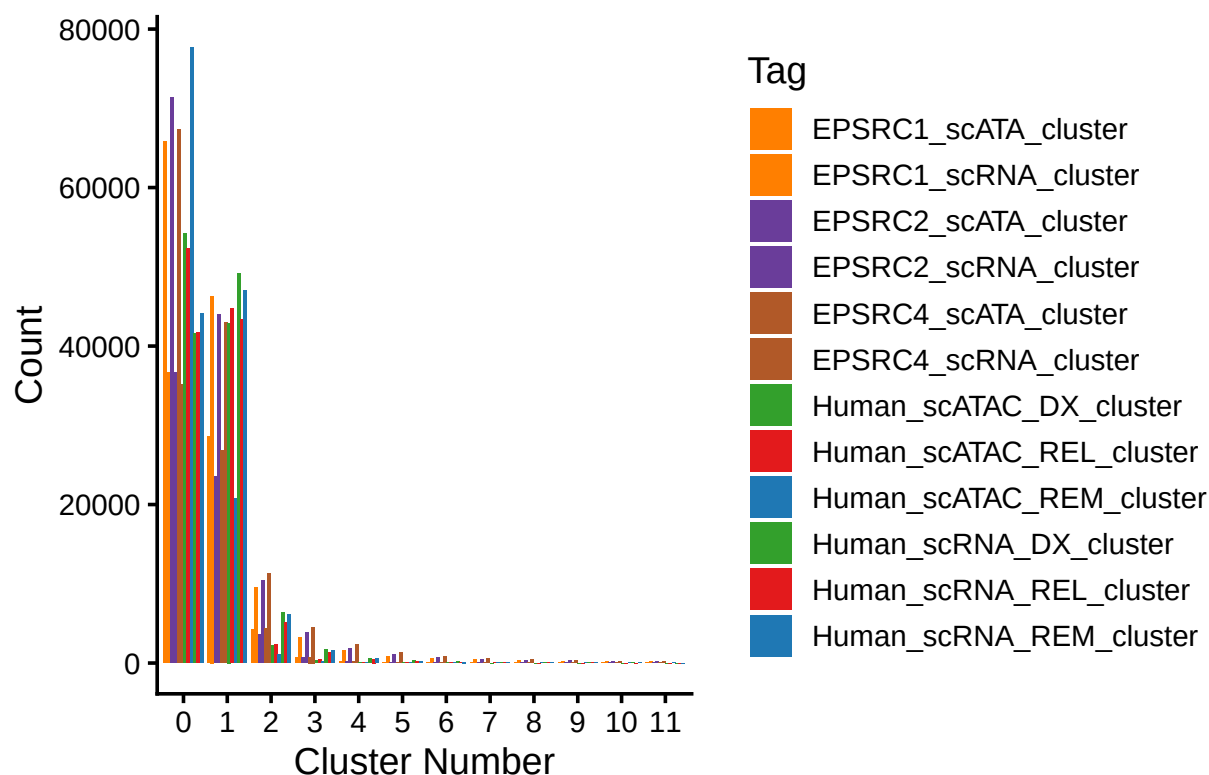
data_merge_cluster <- do.call(rbind, allDataList_cluster)

plot(table(data_merge_cluster$clusterNum))
```



```
ggplot(data_merge_cluster, aes(x = factor(clusterNum), fill = Tag)) +
  geom_bar(position = "dodge") +
  theme_classic(base_size = 14) +
  scale_fill_manual(
    values = c(
      "Human_scATAC_DX_cluster" = "#33a02c",
      "Human_scRNA_DX_cluster" = "#33a02c",
      "Human_scATAC_REM_cluster" = "#1f78b4",
      "Human_scRNA_REM_cluster" = "#1f78b4",
      "Human_scATAC_REL_cluster" = "#e31a1c",
      "Human_scRNA_REL_cluster" = "#e31a1c",
      "EPSRC1_scATA_cluster" = "#ff7f00",
      "EPSRC1_scRNA_cluster" = "#ff7f00",
      "EPSRC2_scATA_cluster" = "#6a3d9a",
      "EPSRC2_scRNA_cluster" = "#6a3d9a",
      "EPSRC4_scATA_cluster" = "#b15928",
      "EPSRC4_scRNA_cluster" = "#b15928"
    )
  ) +
  labs(
    title = "Cluster Number Distribution by Tag",
    x = "Cluster Number",
    y = "Count",
    fill = "Tag"
  )
```

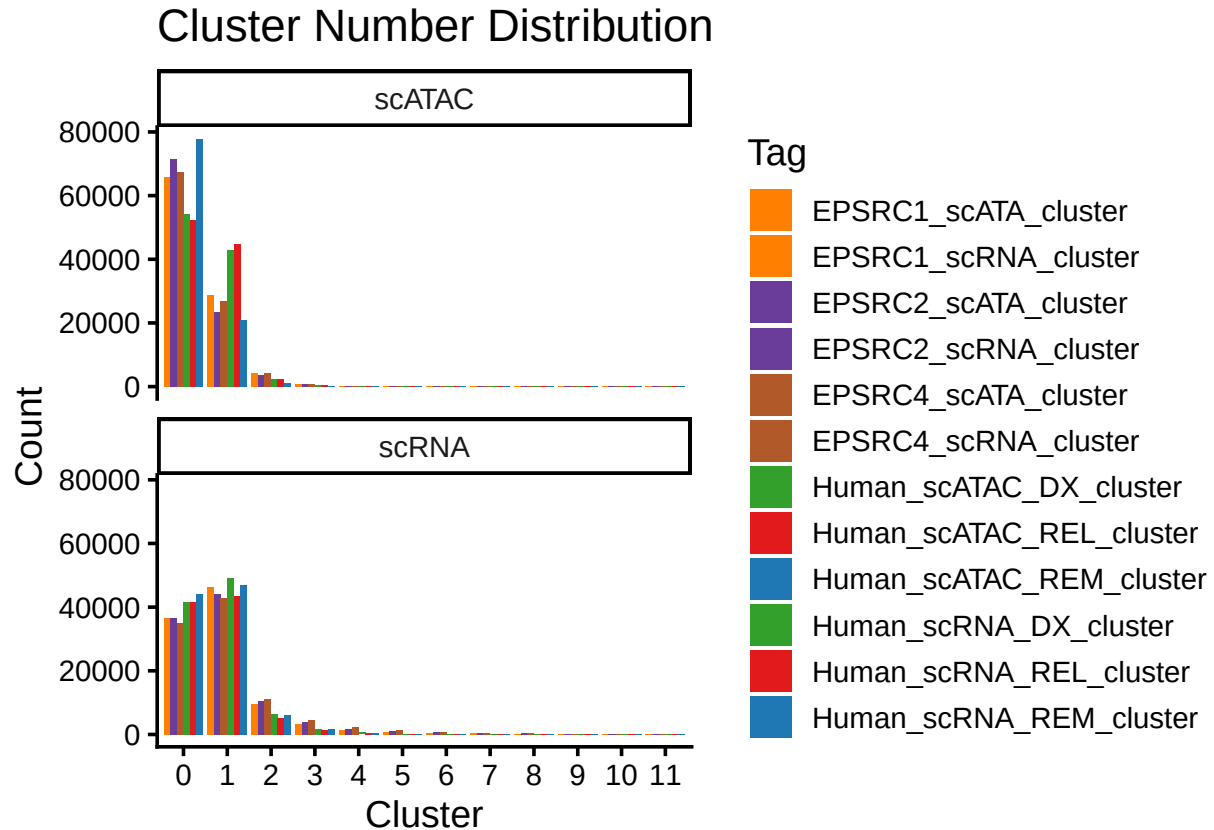
Cluster Number Distribution by Tag



```
data_merge_cluster$modality <- ifelse(grepl("scRNA", data_merge_cluster$Tag), "scRNA", "scATAC")

p_cluster_est <- ggplot(data_merge_cluster, aes(x = factor(clusterNum), fill = Tag)) +
  geom_bar(position = "dodge") +
  #geom_bar(position = "fill") +
  facet_wrap(~ modality, ncol = 1) +
  theme_classic(base_size = 14) +
  scale_fill_manual(values = c(
    "Human_scATAC_DX_cluster" = "#33a02c",
    "Human_scRNA_DX_cluster" = "#33a02c",
    "Human_scATAC_REM_cluster" = "#1f78b4",
    "Human_scRNA_REM_cluster" = "#1f78b4",
    "Human_scATAC_REL_cluster" = "#e31a1c",
    "Human_scRNA_REL_cluster" = "#e31a1c",
    "EPSRC1_scATA_cluster" = "#ff7f00",
    "EPSRC1_scRNA_cluster" = "#ff7f00",
    "EPSRC2_scATA_cluster" = "#6a3d9a",
    "EPSRC2_scRNA_cluster" = "#6a3d9a",
    "EPSRC4_scATA_cluster" = "#b15928",
    "EPSRC4_scRNA_cluster" = "#b15928"
  )) +
  labs(title = "Cluster Number Distribution", x = "Cluster", y = "Count")

p_cluster_est
```



```
data_prop <- data_merge_cluster %>%
  group_by(Tag, clusterNum) %>%
  summarise(count = n(), .groups = "drop") %>%
  group_by(Tag) %>%
  mutate(prop = count / sum(count)) %>%
  ungroup()

data_prop$modality <- ifelse(grepl("scRNA", data_prop$Tag), "scRNA", "scATAC")

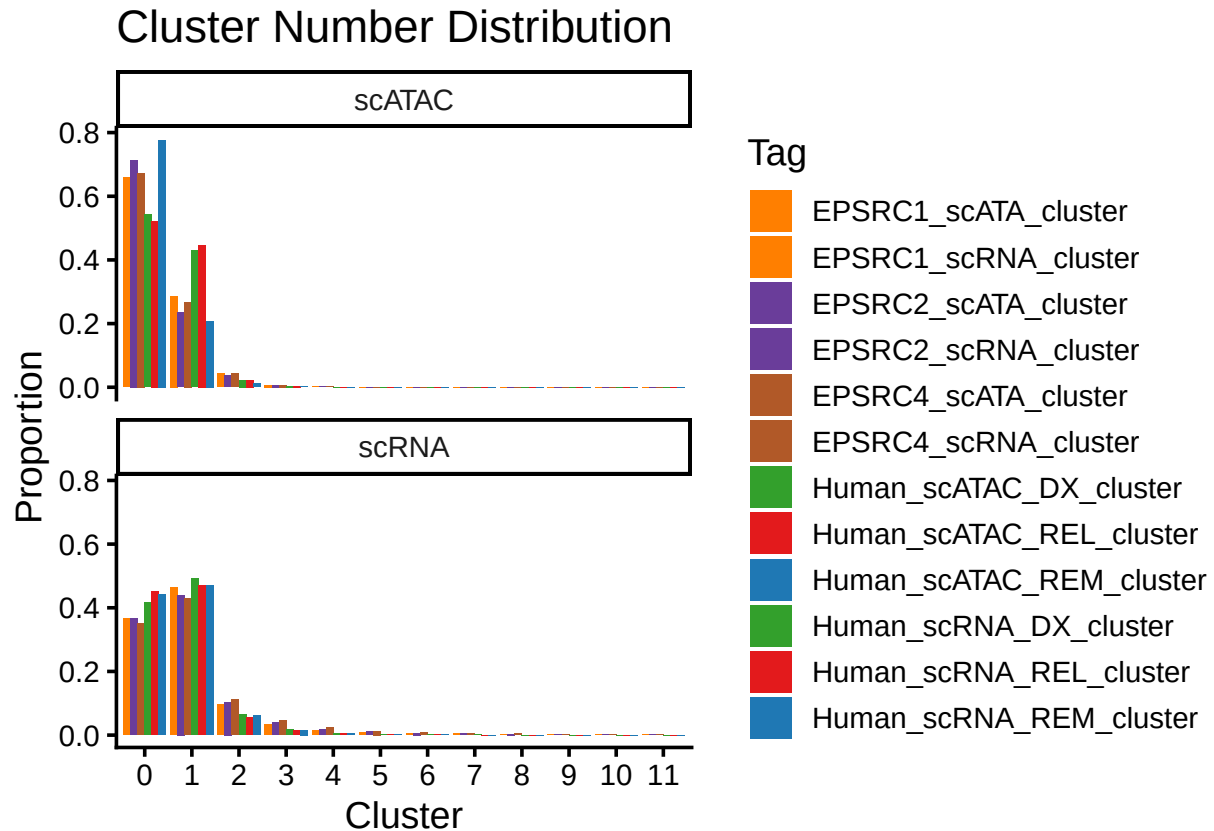
p_cluster_est_prop <- ggplot(data_prop, aes(x = factor(clusterNum), y = prop, fill = Tag)) +
  geom_bar(stat = "identity", position = "dodge") +
  facet_wrap(~ modality, ncol = 1) +
  theme_classic(base_size = 14) +
  scale_fill_manual(values = c(
    "Human_scATAC_DX_cluster" = "#33a02c",
    "Human_scRNA_DX_cluster" = "#33a02c",
    "Human_scATAC_REM_cluster" = "#1f78b4",
    "Human_scRNA_REM_cluster" = "#1f78b4",
    "Human_scATAC_REL_cluster" = "#e31a1c",
    "Human_scRNA_REL_cluster" = "#e31a1c",
    "EPSRC1_scATA_cluster" = "#ff7f00",
    "EPSRC1_scRNA_cluster" = "#ff7f00",
    "EPSRC2_scATA_cluster" = "#6a3d9a",
    "EPSRC2_scRNA_cluster" = "#6a3d9a",
  ))
```

```

    "EPSRC4_scATA_cluster" = "#b15928",
    "EPSRC4_scRNA_cluster" = "#b15928"
  )) +
  labs(title = "Cluster Number Distribution", x = "Cluster", y = "Proportion")

p_cluster_est_prop

```



```

set.seed(2025)
nrow <- 103
ncol <- 225

cols <- c("red", "black", "gray", "white")

prob_red <- 0.05
prob_black <- 0.7
prob_grey <- 0.1
prob_white <- 1-prob_red-prob_black-prob_grey
probs <- c(prob_red, prob_black, prob_grey, prob_white)

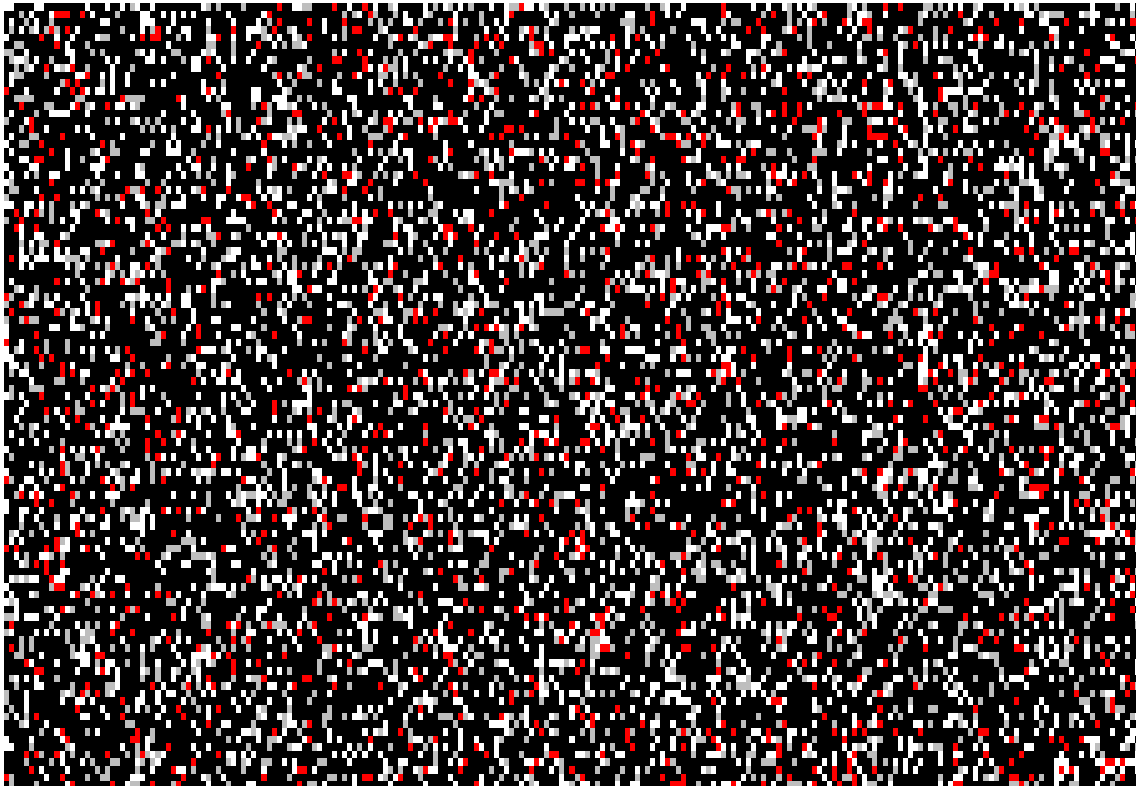
color_vec <- sample(cols, size = nrow * ncol, replace = TRUE, prob = probs)
mat_col <- matrix(color_vec, nrow = nrow, ncol = ncol)

df <- expand.grid(row = 1:nrow, col = 1:ncol)
df$fill <- as.vector(mat_col)

```

```
df$yplot <- nrow - df$row + 1

p <- ggplot(df, aes(x = col, y = yplot, fill = fill)) +
  geom_raster() +
  scale_fill_identity() +
  theme_void() +
  theme(legend.position = "none",
        plot.margin = unit(rep(0,4), "pt"))
p
```



```
# beta distribution
# parameter
n <- 20 # total times
alpha <- 2 # alpha
beta <- 5 # Beta
x <- 0:n # success times

p_bb <- dbetabinom.ab(x, size = n, shape1 = alpha, shape2 = beta)

p_binom <- dbinom(x, size = n, prob = alpha/(alpha + beta))

df <- data.frame(
  x = x,
  BetaBinomial = p_bb,
```

```

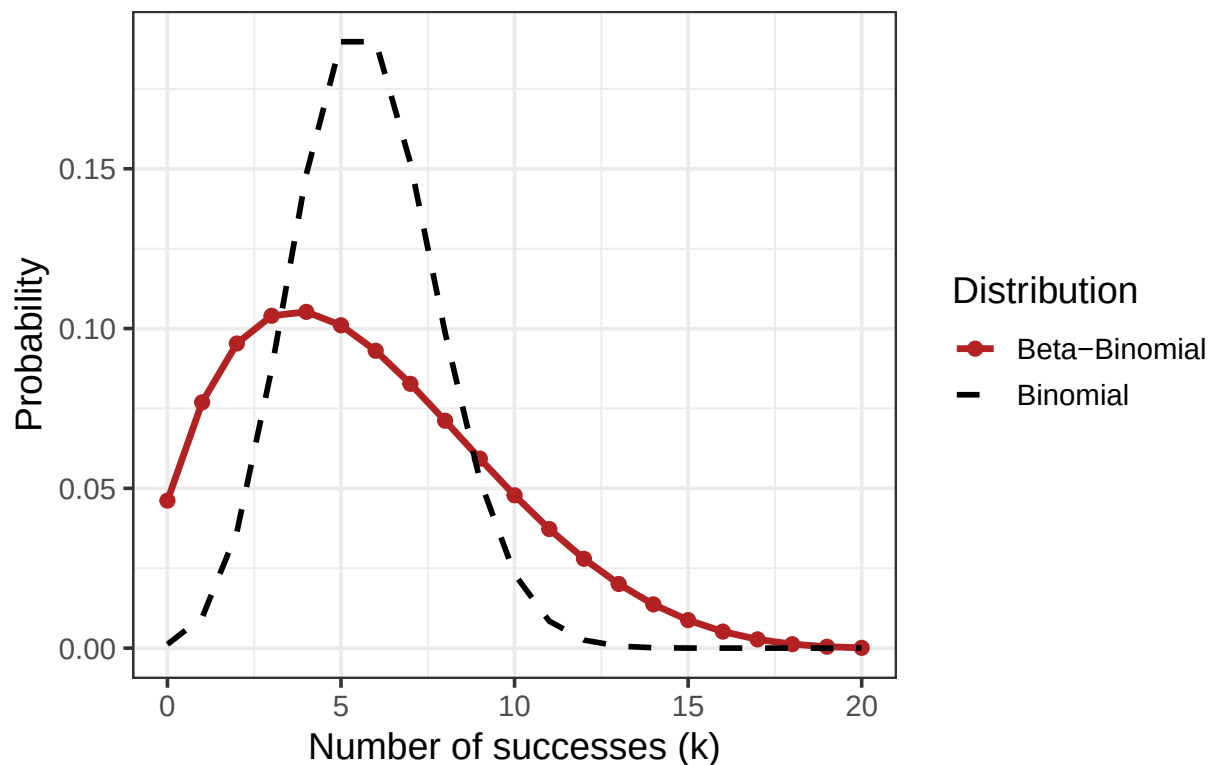
  Binomial = p_binom
)

p_betaDis <- ggplot(df, aes(x = x)) +
  geom_line(aes(y = BetaBinomial, color = "Beta-Binomial"), size = 1.2) +
  geom_point(aes(y = BetaBinomial, color = "Beta-Binomial"), size = 2) +
  geom_line(aes(y = Binomial, color = "Binomial"), linetype = "dashed", size = 1) +
  scale_color_manual(values = c("Beta-Binomial" = "firebrick", "Binomial" = "black")) +
  labs(
    title = bquote("Beta-Binomial vs Binomial (n = " ~ .(n) ~
      ", alpha = " ~ .(alpha) ~ ", beta = " ~ .(beta) ~ ")"),
    x = "Number of successes (k)",
    y = "Probability",
    color = "Distribution"
  ) +
  theme_bw(base_size = 14)
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.

p_betaDis

```

Beta-Binomial vs Binomial (n = 20 , alpha = 2 , beta =



10 sessionInfo

```
sessionInfo()
## R version 4.5.1 (2025-06-13)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.2
##
## Matrix products: default
## BLAS: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK v
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Europe/Stockholm
## tzcode source: internal
##
## attached base packages:
## [1] splines stats4 stats graphics grDevices utils datasets
## [8] methods base
##
## other attached packages:
## [1] extraDistr_1.10.0 dplyr_1.1.4 VGAM_1.1-13 goftest_1.2-3
## [5] fitdistrplus_1.2-4 survival_3.8-3 MASS_7.3-65 patchwork_1.3.2
## [9] reshape2_1.4.4 ggplot2_4.0.0
##
## loaded via a namespace (and not attached):
## [1] Matrix_1.7-4 gtable_0.3.6 crayon_1.5.3 compiler_4.5.1
## [5] tidyselect_1.2.1 Rcpp_1.1.0 stringr_1.5.2 dichromat_2.0-0.1
## [9] scales_1.4.0 yaml_2.3.10 fastmap_1.2.0 lattice_0.22-7
## [13] R6_2.6.1 plyr_1.8.9 labeling_0.4.3 generics_0.1.4
## [17] knitr_1.50 tibble_3.3.0 pillar_1.11.1 RColorBrewer_1.1-3
## [21] rlang_1.1.6 stringi_1.8.7 xfun_0.53 S7_0.2.0
## [25] cli_3.6.5 withr_3.0.2 magrittr_2.0.4 digest_0.6.37
## [29] grid_4.5.1 rstudioapi_0.17.1 lifecycle_1.0.4 vctrs_0.6.5
## [33] evaluate_1.0.5 glue_1.8.0 farver_2.1.2 rmarkdown_2.30
## [37] tools_4.5.1 pkgconfig_2.0.3 htmltools_0.5.8.1
```